



Resumen Clase 11/09 - Sincronización de Procesos

1. Repaso

Procesos vs Thread

Procesos

Thread

Concurrencia vs. Paralelismo

Concurrencia

Paralelismo

Race Condition (condiciones de carrera)

Definición

Causas

2. Atomicidad

Definición

Ejemplo de pérdida de actualizaciones

Traza

Variables atómicas

Primitivas atómicas

3. Exclusión mutua y Sección Crítica

¿Qué es una sección crítica?

Exclusión Mutua

Requisitos de una solución correcta

4. Semáforos

Definición y funcionamiento

Operaciones

`wait()`

`signal()`

Tipos de semáforos

Semáforo binario (mutex)

Semáforo contador

Usos de un semáforo

Cosas a tener en cuenta

5. Patrones de Sincronización

Mutex (Exclusión mutua)

Signaling (Señalización)

Rendezvous (Encuentro / Punto de sincronización)

Barrera (Barrier / Turnstile)

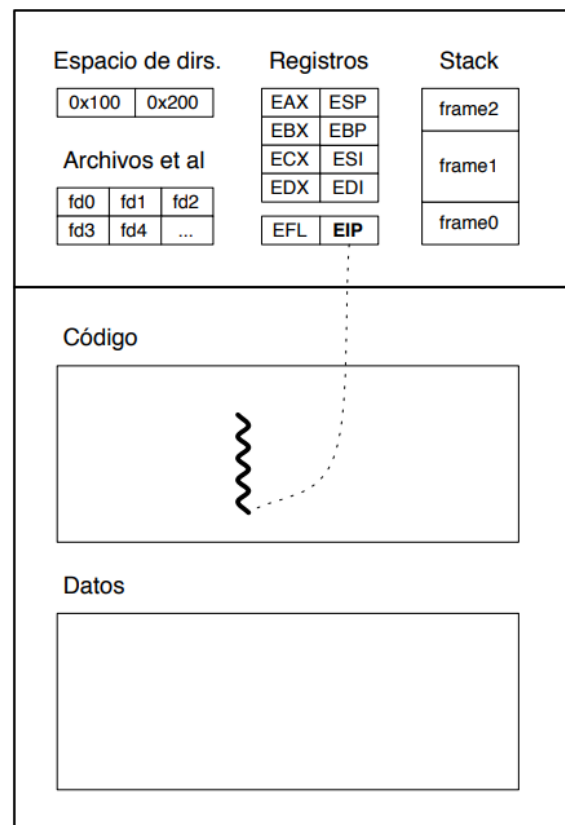
Barrera Reutilizable

1. Repaso

Procesos vs Thread

Procesos

- **Definición:** Un proceso es un programa en ejecución, con su propio espacio de direcciones, recursos y estado.
- **Características:**
 - Tiene un **contexto de ejecución** (registros, PC, pila, etc.).
 - Posee un espacio de memoria independiente (código, datos, stack).
 - Se identifica con un **PID**.



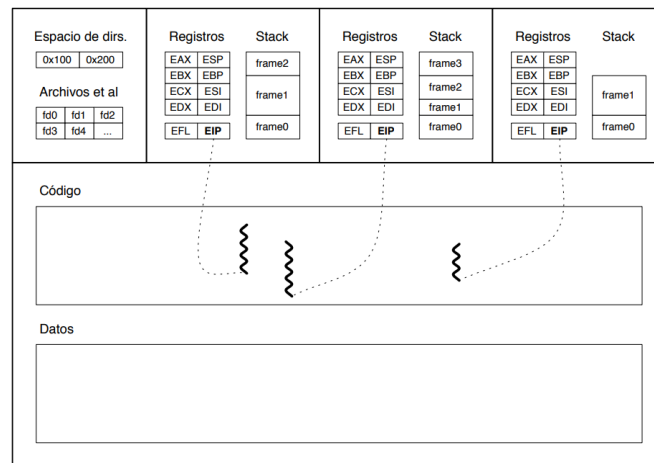
Se pueden tener varios procesos corriendo *concurrentemente*

Thread

- **Definición:** Son "subprocesos" dentro de un mismo proceso.
- **Características:**
 - Ligeros en comparación con los procesos.

Comparten el espacio de direcciones del proceso, pero cada uno tiene su propio stack y registros.

- Permiten **conurrencia dentro de un mismo proceso**.
- Son más baratos de crear y cambiar de contexto que un proceso.



Concurrencia vs. Paralelismo

Concurrencia

- Dos procesos se consideran concurrentes si **no se puede determinar de antemano** el orden exacto de ejecución de sus eventos.
- Depende del **scheduler**.
- Puede ocurrir incluso en un **procesador único** (intercalando la ejecución).

Paralelismo

- Dos procesos ejecutan literalmente **al mismo tiempo** en procesadores distintos.
- Requiere **múltiples núcleos o CPUs**.

Race Condition (condiciones de carrera)

Definición

- Defecto de un programa concurrente por el cual su correctitud depende del orden de ejecución de ciertos eventos.

Causas

- En general los procesos tienen sus variables locales que otros procesos no pueden ver ni acceder. Pero a veces tenemos variables compartidas entre procesos/threads.
 - ¿Que sucede al mezclar concurrencia y memoria compartida? Problemas de sincronización al hacer escrituras, actualizaciones o lecturas concurrentes.
-

2. Atomicidad

Definición

- Una operación es atómica cuando es indivisible. Es decir, cuando no puede ser interrumpida por el procesador hasta terminar.
- A nivel de programación concurrente, una operación atómica garantiza que **ningún otro proceso/thread puede observar un estado intermedio**.
- En general no sabemos qué operaciones se realizan en un paso y cuáles pueden ser interrumpidas.

Ejemplo de pérdida de actualizaciones

- Sean A y B dos instancias en ejecución del mismo programa 'Suma'. Ambas corren concurrentemente y comparten una variable x. ¿Cuál es la salida esperada si el código que corren es el siguiente?
- Memoria compartida `x = 0`

```
Suma ():  
    x = x + 1;  
    print(x);
```

- Pensemos qué pasa a nivel de máquina.
- Esta operación sobre memoria compartida involucra:
 1. Leer valor de x
 2. Operar (sumarle 1 a este valor)

3. 3 Escribir nuevo valor en x

- El scheduler puede interrumpir la operación en medio de alguna de estas 3 partes.

Traza

Proceso A	Proceso B	Estados
lee x		x = 0
suma 1 a x		x = 0
	lee x	x = 0
almacena 1 en x		x = 1
	suma 1 a x	x = 1
	almacena 1 en x	x = 1

Variables atómicas

- Una variable atómica es un objeto que nos permite realizar operaciones de escritura y lectura de forma atómica.
- Para interactuar con esas variables es necesario utilizar primitivas atómicas.

Primitivas atómicas

- `getAndInc()` : Devuelve el entero atómico sumado 1.
- `getAndAdd()` : Devuelve el entero atómico sumado la cantidad especificada.
- `set(unsigned int value)` : Asigna al objeto un valor pasado por parámetro.

3. Exclusión mutua y Sección Crítica

¿Qué es una sección crítica?

- **Definición:**

La **sección crítica** es la parte de un programa donde se accede a **memoria compartida** (lectura o escritura) y necesitamos que la ejecución sea **atómica**.

- **Problema:**

Si dos procesos acceden a la misma sección crítica al mismo tiempo, se pueden generar **inconsistencias**.

Exclusión Mutua

- **Definición:**

La **exclusión mutua** garantiza que **sólo un proceso a la vez** puede estar ejecutando una sección crítica.

- **Objetivo:** evitar condiciones de carrera.

- **Herramientas para lograrlo:**

- Operaciones atómicas (ej. `TestAndSet`, `Swap`).
- **Semáforos (mutex).**
- Monitores (tema más avanzado).

Requisitos de una solución correcta

Toda solución al problema de la sección crítica debe cumplir con:

1. **Exclusión mutua:**

No hay dos procesos simultáneamente en la sección crítica.

2. **Progreso:**

Si ningún proceso está en la sección crítica, algún proceso debe poder entrar.

3. **Espera limitada (bounded waiting):**

Cada proceso tiene un límite en la cantidad de veces que otros pueden entrar antes que él. (Evita la **inanición**).

4. Semáforos

Definición y funcionamiento

- Un **semáforo** es un **tipo abstracto de datos** que permite **controlar el acceso** de múltiples procesos a un recurso compartido.

- Internamente mantiene un **valor entero**, pero **no es accesible directamente**.
- La única forma de interactuar con un semáforo es mediante dos operaciones atómicas:
 - `wait()` (también llamada `P`, `down`)
 - `signal()` (también llamada `V`, `up`)

Operaciones

`wait()`

- Si el valor del semáforo es > 0 → lo decrementa y el proceso continúa.
- Si el valor es ≤ 0 → el proceso **se bloquea** hasta que otro haga un `signal()`.

`signal()`

- Incrementa el valor del semáforo.
- Si había procesos bloqueados en `wait()`, despierta a uno de ellos.

Tipos de semáforos

Semáforo binario (mutex)

- Valor inicial: `1`.
- Se utiliza para asegurar **exclusión mutua**.

Semáforo contador

- Valor inicial: `N`.
- Permite hasta `N` procesos concurrentes accediendo al recurso.
- Ejemplo: controlar el acceso a un pool de conexiones.

Usos de un semáforo

1. Exclusión mutua

- Garantizar que una sección crítica solo sea ejecutada por un proceso a la vez.

2. Sincronización

- Asegurar que un proceso ejecute cierta acción **solo después** de que otro haya hecho algo.

Cosas a tener en cuenta

- No podemos consultar el valor de un semáforo directamente.
- No sabemos **qué proceso se despierta** en un `signal()` (depende de la implementación).
- El orden puede afectar el rendimiento o provocar **inanición**, pero no la corrección.
- Si se usan mal, los semáforos pueden producir **deadlocks**.



Un **deadlock** (o interbloqueo) ocurre cuando un conjunto de procesos queda **bloqueado indefinidamente**, porque cada uno está esperando un recurso que nunca será liberado.

En otras palabras:

- **Ningún proceso puede avanzar.**
- Todos esperan por otros, y la espera nunca se resuelve.

5. Patrones de Sincronización

Mutex (Exclusión mutua)

- **Objetivo:** garantizar que solo un proceso a la vez ejecute una sección crítica.
- **Herramienta:** semáforo binario (inicializado en 1).

```
sem mutex = sem(1);
```

```
Proceso() {  
    while (true) {  
        // sección no crítica  
        mutex.wait();    // entrada
```



```

    // sección crítica
    mutex.signal(); // salida
    // sección no crítica
}
}

```

Signaling (Señalización)

- **Objetivo:** asegurar que cierta acción de un proceso ocurra antes que otra acción de otro proceso.
- **Ejemplo:** A debe ejecutar `A1()` antes que B ejecute `B2()`.

```
sem permisoB = sem(0);
```

Proceso A:

```

A1();
permisoB.signal(); // habilita a B
A2();

```

Proceso B:

```

B1();
permisoB.wait(); // espera a que A1 termine
B2();

```

Rendezvous (Encuentro / Punto de sincronización)

- **Objetivo:** dos procesos deben llegar a un punto antes de poder continuar.
- **Ejemplo:** A1 y B1 deben ejecutarse antes de A2 y B2.

```

sem permisoA = sem(0);
sem permisoB = sem(0);

```

Proceso A:

```

A1();
permisoB.signal(); // habilita a B

```

```
permisoA.wait(); // espera a B
A2();
```

Proceso B:

```
B1();
permisoA.signal(); // habilita a A
permisoB.wait(); // espera a A
B2();
```

Barrera (Barrier / Turnstile)

- **Objetivo:** todos los procesos deben llegar a cierto punto antes de continuar.
- **Ejemplo:** N estudiantes implementan, y recién cuando todos terminan pueden experimentar.

```
sem barrera = sem(0);
sem mutex = sem(1);
int counter = 0;
```

```
Proceso Estudiante() {
    implementarTp();
```

```
    mutex.wait();
    counter++;
    if (counter == N) barrera.signal(); // el último abre la barrera
    mutex.signal();
```

```
    barrera.wait(); // todos esperan
    barrera.signal(); // vuelve a abrir para los demás
```

```
    experimentar();
}
```

(Esto se llama "turnstile" porque funciona como un molinete que deja pasar de a uno y luego se cierra).

Barrera Reutilizable

- **Objetivo:** repetir la sincronización varias veces (ej: varias rondas de implementación/experimentación).
- Se logra combinando la barrera con un mecanismo de **reinicio** (más avanzado, se ve después).